

Proyecto: AutoML Distribuido (Local Training / Remote LLM)
Hardware: Servidor 32 GB (Ollama) | PC Local (Ejecución)

Esquema

Etapa	Nombre	Responsable	Salida
Etapa 0	Data Discovery & Target Definition	Usuario	- Features - Features blindadas. - Targets
Etapa 1	Augmented Intelligence Profiling	PC Local	TOON dossier - Tipos de datos - Niveles de entropía
Etapa 2	Intelligent Pipeline Orchestration	Servidor (LLM)	Script Python - Código ejecutable - Contenerización - Bucle de autorreparación

ETAPA 0: Data Discovery & Target Definition

1. Carga del Dataset

El usuario carga el archivo de datos a través de la interfaz. El sistema valida que el archivo sea legible y contabiliza el número total de filas N, que se utilizará en los muestreos posteriores.

- Formatos soportados:** CSV, Excel (.xlsx), Parquet.
- Acción:** validar la integridad del archivo y contar las filas antes de cualquier procesamiento.

Importante: Si el archivo no puede parsearse, mostrar un error claro y abortar la Etapa 0 antes de continuar.

2. Definición de Roles (Features vs. Targets)

- Acción:** interfaz en la que el usuario selecciona qué columnas "predicen" (features) y cuáles "son predichas" (targets).
- Importancia:** Esto es vital para el Paso 3 de la siguiente etapa (Dependencia de Targets), ya que el sistema solo calculará correlaciones entre las columnas marcadas como *Target*.

Consejo: Puede ser más cómodo almacenar el manifiesto de roles como un diccionario tipado: {'features': [...], 'targets': [...], 'shielded': [...]}

3. Declaración de "Features Blindadas"

El usuario puede marcar columnas de tipo feature como "Intocables". Estas columnas se incluirán en el modelo independientemente de su relevancia estadística.

- **Lógica técnica:** Si una variable tiene una importancia de 0.001 pero está en la lista de "blindadas", el dossier que se envíe al LLM llevará una etiqueta especial: `mandatory: True`.
- **Efecto en el LLM:** El LLM recibirá la instrucción: *"Esta variable es estadísticamente irrelevante, pero el usuario exige su inclusión por razones de negocio. Genere el código que la incluya"*.

Instrucción inyectada al LLM para features blindadas:

"This feature is statistically irrelevant, but the user requires its inclusion for business reasons. Generate code that includes it in every stage of the pipeline."

Esta instrucción se inyecta en el Paso 1.4, durante el ensamblaje del payload del TOON que se envía al LLM. Para cada columna con `user_mandatory: True`, se añade el campo `llm_instruction` al bloque de esa columna:

4. Declaración de los hiperparámetros de búsqueda

El usuario configura la estrategia de búsqueda de hiperparámetros. Puede elegir un método concreto o delegar la decisión en el LLM orquestador.

Campo	Tipo	Descripción
<code>search_strategy</code>	<code>auto</code> <code>grid</code> <code>random</code> <code>optuna</code>	<i>Si "auto", el LLM elige según el contexto estadístico del TOON</i>
<code>max_iter</code>	<code>int</code> (default: 50)	<i>Número máximo de combinaciones evaluadas (random / optuna)</i>
<code>max_combinations</code>	<code>int</code> (default: 200)	<i>Límite de combinaciones para grid search</i>
<code>cv_folds</code>	<code>int</code> (default: 10)	<i>Número de folds de validación cruzada</i>
<code>timeout_minutes</code>	<code>int</code> (default: 30)	<i>Tiempo máximo total de búsqueda en minutos</i>

5. Creación del fichero manifest

El fichero manifest con la información obtenida en esta etapa no se almacena en disco. Se construye en Etapa 0 y se pasa como argumento entre funciones a lo largo de toda la ejecución.

```
manifest = {
  'features' : ['departamento', 'antigüedad', 'turno'],
  'targets' : ['sala', 'prioridad'],
  'shielded' : ['turno'], # features blindadas
  'execution_mode' : 'auto' | 'docker' | 'subprocess' #FOR STAGE 3.5
  'search_config' : { #FOR STAGE 3.4 o 3.3
    'search_strategy' : 'auto', # 'auto' | 'grid' | 'random' | 'optuna'
    'max_iter' : 50,
    'max_combinations' : 200,
    'cv_folds' : 10,
    'timeout_minutes' : 30,
  }
}
```

ETAPA 1: Categorical Semantic Analysis (TOON Dossier)

Esta etapa realiza un análisis estadístico completo del conjunto de datos. Se ejecuta íntegramente en la máquina local (el LLM no interviene hasta el Paso 1.4). El resultado es el TOON Dossier, que el LLM orquestador consume en la etapa 2.

1. Cálculo de Niveles (Umbral del 5%):

Para cada columna se calcula el ratio de unicidad $R = \text{nunique} / \text{num_rows}$ y se asigna un nivel de tipo:

Nivel	Nombre	Condición	Sub-clasificacion
Tipo 1	Binaria	$\text{nunique} == 2$	-
Tipo 2	Categórica	$R < 0.05$	Nominal → preguntar LLM Ordinal → preguntar LLM
Tipo 3	Numérica	$0.05 \leq R < 0.8$	Discreta → dtype=int Continua → dtype=float
Tipo 4	Alta Cardinalidad	$R \geq 0.80$	Identificador/texto → IG bajo

2. Manejo de NaNs e Imputación Temporal para el IG

Los estimadores de información mutua de sklearn fallan al detectar valores NaN. Se aplica una imputación temporal a una copia de cada columna, usando la estrategia más robusta según el tipo:

- Categóricas/Binarias (Tipo 1-2): moda (valor único representativo discreto).
- Numéricas (Tipo 3-4): mediana. Se prefiere sobre la media porque en este punto aún no se conoce la distribución de la columna (la mediana es robusta frente a valores atípicos y a la asimetría).

```
def impute_for_ig(series: pd.Series, tipo: int) -> pd.Series:
    """
    Imputación temporal para cálculo de IG. Nunca modifica el dataset
    original – opera sobre una copia.
    """
    s = series.copy()
    if tipo in (1, 2): # Binaria / Categórica → moda
        return s.fillna(s.mode()[0])
    else: # Numérica / Alta Cardinalidad → mediana
        return s.fillna(s.median())

def get_null_ratio(series: pd.Series) -> float:
    """
    Registra el porcentaje de NaNs para incluirlo en el TOON.
    """
    return round(series.isna().mean(), 4)
```

Importante:

El `null_ratio` se añade al bloque de cada columna en el TOON (ver el Paso 1.5). El LLM orquestador usará este valor para decidir si aplicar imputación, eliminación de columna o un indicador de missingness en el pipeline generado.

3. Cálculo de la entropía (Information Gain) por par:

Se usan los estimadores de información mutua de scikit-learn. La elección de función depende del tipo de la feature y del target:

Tipo Target	Tipo Feature	Función
Tipo 1 o 2 (Cat)	Tipo 1 o 2 (Cat)	<code>mutual_info_classif(..., discrete_features=True)</code>
Tipo 1 o 2 (Cat)	Tipo 3 o 4 (Num)	<code>mutual_info_classif(..., discrete_features=False)</code>
Tipo 3 o 4 (Num)	Tipo 1 o 2 (Cat)	<code>mutual_info_regression(..., discrete_features=True)</code>
Tipo 3 o 4 (Num)	Tipo 3 o 4 (Num)	<code>mutual_info_regression(..., discrete_features=False)</code>

Importante: `mutual_info_regression` utiliza estimadores de k vecinos (Kruskal) sensibles a distribuciones muy asimétricas. Se aplica una transformación automática a una copia de la columna (el dataset original nunca se modifica). La transformación aplicada se debe registrar en el TOON

```
from scipy.stats import skew
from sklearn.preprocessing import PowerTransformer

def auto_transform_for_ig(df: pd.DataFrame, col: str) -> tuple[pd.Series, str]:
    """
    Applies a normalising transform before the IG estimator.
    Works on a copy only – the original DataFrame is never modified.
    Returns (transformed_series, transform_name).
    """
    serie = df[col].copy().dropna()
    sk = abs(skew(serie))
    if sk > 1.0 and serie.min() >= 0:      # Right-skewed, non-negative
        return np.log1p(df[col].copy()), 'log1p'
    elif sk > 1.0:                        # Skewed, may contain negatives
        pt = PowerTransformer(method='yeo-johnson')
        vals = pt.fit_transform(df[[col]].copy()).flatten()
        return pd.Series(vals, index=df.index), 'yeo-johnson'
    return df[col].copy(), 'none'         # No transform needed

def compute_ig_pair(df, feature_col, target_col,
                    tipo_feature, tipo_target) -> tuple[float, str]:
    """Returns (ig_score, transform_applied)."""

    # 1. Impute NaNs temporarily
    feat = impute_for_ig(df[feature_col], tipo_feature)
    tgt = impute_for_ig(df[target_col], tipo_target)

    # 2. Apply skewness transform if needed (numeric features only)
    transform = "none"
    if tipo_feature in (3, 4):
```

```

    feat, transform = auto_transform_for_ig( pd.DataFrame({feature_col:
                                                         feat}), feature_col)

# 3. Select the correct MI estimator
discrete = tipo_feature in (1, 2)
if tipo_target in (1, 2):
    score = mutual_info_classif( feat.values.reshape(-1,1), tgt,
                                discrete_features=discrete, random_state=42)[0]
else:
    score = mutual_info_regression( feat.values.reshape(-1,1), tgt,
                                   discrete_features=discrete, random_state=42)[0]
return round(float(score), 6), transform

```

Skewness	Valores	Transformación
>1.0	≥ 0	$\log 1p$
> 1.0	cualquier	<i>yeo-johnson (compatible con negativos)</i>
≤ 1.0	cualquier	<i>none - no necesita transformación</i>

Consejo:

Si solo hay un *target* se debe aplicar el IG entre todas las *features* elegidas y el *target*.

$IG(f_1, target)$, $IG(f_2, target)$, ..., $IG(f_n, target_n)$

Si hay *N targets*, se debe aplicar IG entre todas las *features* y los *targets*.

$IG(f_1, target_1)$, $IG(f_1, target_2)$, $IG(f_1, target_n)$, ..., $IG(f_{n-1}, target_n)$, $IG(f_n, target_n)$

4. Construcción de la muestra estratificada (n = 100)

Solo se ejecuta si se han detectado *features* categóricas. Produce una muestra de exactamente 100 filas representativa del dataset, que el LLM semántico del Paso 1.4 usará para clasificar variables como nominales u ordinales. Casuísticas:

- 1 target: estratificación directa a lo largo de la columna target.
- Múltiples targets: se crea una columna temporal combinada (p.ej., sala_prioridad = "H_A") y se estratifica por ella. La columna combinada nunca se incluye en el TOON.

SALA	PRIORIDAD	SALA_PRIORIDAD
H	A	H_A
H	B	H_B
NH	B	NH_B

El script debe repartir las 100 filas proporcionalmente:

- Si el grupo **H_A** representa el 1% del dataset, el sistema se asegura de que **al menos 1** de las 100 filas que recibe el LLM sea un ejemplo real de una sala H con prioridad A.
- Si el grupo **NH_B** representa el 60%, el LLM recibirá unas 60 filas de dicho grupo.
- No se incluye la columna combinada en el dossier para el LLM. Se usa solo para calcular la representatividad.

- **Nueva implementación:** la versión anterior usaba `clip(lower=1)`, que puede inflar la muestra si hay muchos grupos minoritarios (p.ej., 80 grupos → 180 filas), lo que distorsiona de forma impredecible el contexto del LLM. **Estrategia adoptada:** mantener siempre `n=100`. Si un grupo no puede representarse sin superar ese límite, se descarta y se documenta en el TOON como `rare_groups_excluded`.

```
def stratified_sample(df: pd.DataFrame, target_cols: list[str],
                    n: int = 100) -> tuple[pd.DataFrame, list]:

    """
    Samples exactly n rows with proportional stratification.
    Returns (sample_df, rare_groups_excluded).
    """

    # 1. Build a stratification column
    if len(target_cols) == 1:
        strata_col = target_cols[0]
    else:
        df = df.copy()
        df['_strata'] = df[target_cols].astype(str).agg('_', join, axis=1)
        strata_col = '_strata'
    # 2. Largest Remainder allocation
    group_sizes = df.groupby(strata_col).size()
    exact_quotas = group_sizes / len(df) * n
    floor_quotas = exact_quotas.floor().astype(int)
    deficit = n - floor_quotas.sum()
    floor_quotas[exact_quotas.sub(floor_quotas).nlargest(deficit).index] += 1

    # 3. Strict strategy: sacrifice rare groups to keep n exact
    rare_excluded = [g for g, q in floor_quotas.items() if q < 1]
    # 4. Sample (skip rare groups)
    parts = [
        grp_df.sample(n=min(floor_quotas[grp], len(grp_df)), random_state=42)
        for grp, grp_df in df.groupby(strata_col)
        if grp not in rare_excluded
    ]
    result = pd.concat(parts)
    if '_strata' in result.columns:
        result = result.drop(columns=['_strata'])
    return result, rare_excluded
```

5. Clasificación semántica LLM — Nominal vs. Ordinal

Solo aplica a columnas de Tipo 2 (categóricas). Se envía el payload TOON a un LLM para determinar si cada variable categórica es Nominal u Ordinal. El argumento `toon_payload` que se pasa al LLM es el dic de la etapa 0, paso 3

El parámetro `num_ctx` define la ventana de contexto total en Ollama, que se comparte entre el input (system prompt + payload) y el output (respuesta generada). Se calcula dinámicamente en tres pasos:

- Techo arquitectural: se consulta a la API de Ollama el límite máximo que soporta el modelo según su arquitectura.
- Techo de VRAM: se estima la memoria disponible en el servidor. El KV-cache consume aproximadamente 200 bytes por token, por lo que `ctx_por_vram = vram_libre × 0.8 / 200`. Si el sistema es CPU-only (sin GPU), este límite no se aplica.

- Tokens necesarios: se estima el input en $n_chars / 4$ (1 token $\approx$ 4 caracteres), más la respuesta esperada. La respuesta se estima dinámicamente en función del número de columnas categóricas: cada columna genera aproximadamente 80 tokens de respuesta (subclass + mapping + reasoning).

```
def estimate_resp_tokens(n_categorical_cols: int) -> int:
    """~80 tokens per column: subclass(10) + mapping(30) + reasoning(40)."""
    return n_categorical_cols * 80

def compute_num_ctx(payload_str, system_prompt,
                    ollama_url, model_name,
                    n_categorical_cols: int) -> int:

    """Computes a safe num_ctx by taking the minimum of:
    (1) what the payload needs,
    (2) the model's architectural maximum,
    (3) what fits in available VRAM."""

    # 1. Architectural ceiling from Ollama API

    info = requests.post(f'{ollama_url}/api/show',
                        json={'name': model_name}).json()
    max_ctx = info['model_info']['llama.context_length']

    # 2. VRAM ceiling (GPU only)
    try:
        ps = requests.get(f'{ollama_url}/api/ps').json()
        vram_used = sum(m['size_vram'] for m in ps.get('models', []))
        smi = subprocess.run(
            ['nvidia-smi', '--query-gpu=memory.total',
             '--format=csv,noheader,nounits'],
            capture_output=True, text=True)
        vram_total_mb = int(smi.stdout.strip().split('\n')[0])
        vram_free = vram_total_mb * 1024 * 1024 - vram_used
        ctx_per_vram = int(vram_free * 0.8 / 200) # 80% of free VRAM
    except Exception:
        ctx_per_vram = max_ctx # CPU-only: no VRAM limit

    # 3. Tokens needed by the payload
    input_tokens = (len(payload_str) + len(system_prompt)) // 4
    resp_tokens = estimate_resp_tokens(n_categorical_cols)
    needed_tokens = input_tokens + resp_tokens
    rounded = ((needed_tokens // 512) + 1) * 512 #round up to nearest 512
    return min(rounded, max_ctx, ctx_per_vram)
```

Con *num_ctx* calculado dinámicamente y *temperature*: 0.1 (valor bajo para forzar respuestas deterministas y precisas), evitando variabilidad innecesaria en la clasificación Nominal/Ordinal:

```
system_prompt = (
    "You are a world-class Data Science Assistant. "
    "Analyze the provided TOON to distinguish between NOMINAL and ORDINAL "
    "features. "
    "Rules:\n"
    "1. Find natural hierarchies in 'unique_pool'.\n"
    "2. Use 'representative_sample' to see behaviour vs targets.\n"
```

```

        "3. If ORDINAL, provide a numeric 'mapping'. If NOMINAL, mapping is
        null.\n"
        "4. Output ONLY the TOON structure. No conversational text."
    )

```

```

user_prompt = f"Process the following dataset profile in TOON
format:\n\n{payload}"

```

```

payload = {
    "model": MODEL, # Specify LLM to be used
    "messages": [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ],
    "stream": False,
    "options": {
        "temperature": 0.1, # Precise and deterministic
        "num_ctx": 4096 # LLM context size. Should be calculated
                        # See compute_num_ctx function
    }
}

```

6. Construcción del JSON

Este paso se realiza durante toda la etapa 1 ya que se basa en la construcción del JSON donde se almacena toda la información que se ha calculado en esta etapa y que será usado para construir el correspondiente TOON que será enviado al LLM (ver Etapa 1.7). A continuación se incluye un ejemplo de JSON par uno y múltiples targets.

JSON TOON - UN SOLO TARGET

```

{
    "global_metadata": {
        "total_rows": 1000000,
        "targets": ["sala"],
        "rare_groups_excluded": [],
        "sampling_strategy": "Stratified_SingleTarget",
        "search_config": {
            "search_strategy": "auto",
            "max_iter": 50,
            "max_combinations": 200,
            "cv_folds": 10,
            "timeout_minutes": 30
        }
    },
    "representative_sample": [
        {"id_emp": 1024, "departamento": "Ventas", "antiguedad": 5, "teletrabajo":
        "Si", "sala": "Sala_A"},
        {"id_emp": 2055, "departamento": "IT", "antiguedad": 2, "teletrabajo":
        "No", "sala": "Sala_B"}
    ],
    "categorical_subclass_evaluation": {
        "departamento": {
            "technical_level": 2,
            "is_target": false,
            "ig_scores": {"sala": 0.38},
            "ig_transform_applied": "none",

```

```

    "null_ratio": 0.02,
    "unique_pool": ["IT", "Ventas", "RRHH", "Finanzas"],
    "user_mandatory": false
  },
  "cargo_nivel": {
    "technical_level": 1,
    "is_target": false,
    "ig_scores": {"sala": 0.12},
    "ig_transform_applied": "none",
    "null_ratio": 0.00,
    "unique_pool": ["Si", "No"],
    "user_mandatory": true,
    "llm_instruction": "This feature is statistically irrelevant, but the user
requires its inclusion for business reasons. Generate code that includes it in
every stage of the pipeline."
  }
},

"target_subclass_evaluation": {
  "sala": {
    "technical_level": 2,
    "is_target": true,
    "ig_scores": {},
    "ig_transform_applied": "none",
    "null_ratio": 0.00,
    "unique_pool": ["H", "NH", "O"],
    "user_mandatory": false
  }
},

"semantic_classification_results": {
  "departamento": {
    "subclass": "NOMINAL",
    "mapping": null,
    "reasoning": "Corporate departments are distinct entities with no natural
hierarchy."
  },
  "teletrabajo": {
    "subclass": "ORDINAL",
    "mapping": {"No": 0, "Si": 1},
    "reasoning": "Boolean-like categorical with a logical progression from
absence to presence."
  },
  "sala": {
    "subclass": "NOMINAL",
    "mapping": null,
    "reasoning": "Room labels are distinct identifiers with no natural
ordering."
  }
}
}

```

JSON TOON - MÚLTIPLES TARGET

```

{
  "global_metadata": {
    "total_rows": 1000000,
    "targets": ["sala", "prioridad"],

```

```

    "rare_groups_excluded": ["NH_C"],
    "sampling_strategy": "Stratified_MultiTarget_Combined",
    "search_config": {
        "search_strategy": "auto",
        "max_iter": 50,
        "max_combinations": 200,
        "cv_folds": 10,
        "timeout_minutes": 30
    }
},

    "representative_sample": [
        {"id_emp": 5021, "departamento": "IT", "cargo_nivel": "Senior", "turno":
"Noche", "sala": "Sala_C", "prioridad": "Alta"},
        {"id_emp": 1092, "departamento": "Ventas", "cargo_nivel": "Junior", "turno":
"Tarde", "sala": "Sala_A", "prioridad": "Baja"}
    ],
    "categorical_subclass_evaluation": {
        "departamento": {
            "technical_level": 2,
            "is_target": false,
            "ig_scores": {"sala": 0.42, "prioridad": 0.05},
            "ig_transform_applied": "none",
            "null_ratio": 0.01,
            "unique_pool": ["IT", "Ventas", "RRHH", "Finanzas"],
            "user_mandatory": false
        },
        "cargo_nivel": {
            "technical_level": 2,
            "is_target": false,
            "ig_scores": {"sala": 0.15, "prioridad": 0.82},
            "ig_transform_applied": "none",
            "null_ratio": 0.00,
            "unique_pool": ["Junior", "Middle", "Senior", "Manager", "Director"],
            "user_mandatory": false
        },
        "turno": {
            "technical_level": 2,
            "is_target": false,
            "ig_scores": {"sala": 0.65, "prioridad": 0.10},
            "ig_transform_applied": "none",
            "null_ratio": 0.00,
            "unique_pool": ["Mañana", "Tarde", "Noche"],
            "user_mandatory": true,
            "llm_instruction": "This feature is statistically irrelevant, but the user
requires its inclusion for business reasons. Generate code that includes it in
every stage of the pipeline."
        }
    },

    "target_subclass_evaluation": {
        "sala": {
            "technical_level": 2,
            "is_target": true,
            "ig_scores": {},
            "ig_transform_applied": "none",
            "null_ratio": 0.00,
            "unique_pool": ["Sala_A", "Sala_B", "Sala_C"],

```

```

        "user_mandatory": false
    },
    "prioridad": {
        "technical_level": 2,
        "is_target": true,
        "ig_scores": {},
        "ig_transform_applied": "none",
        "null_ratio": 0.00,
        "unique_pool": ["Baja", "Media", "Alta", "Critica"],
        "user_mandatory": false
    }
},
"semantic_classification_results": {
    "departamento": {
        "subclass": "NOMINAL",
        "mapping": null,
        "reasoning": "Department names have no inherent mathematical order."
    },
    "cargo_nivel": {
        "subclass": "ORDINAL",
        "mapping": {"Junior": 0, "Middle": 1, "Senior": 2, "Manager": 3,
"Director": 4},
        "reasoning": "Professional hierarchy detected. High IG for 'prioridad'
confirms structured influence."
    },
    "turno": {
        "subclass": "ORDINAL",
        "mapping": {"Mañana": 0, "Tarde": 1, "Noche": 2},
        "reasoning": "Temporal sequence detected. High IG for 'sala' suggests
shift-based allocation."
    },
    "sala": {
        "subclass": "NOMINAL",
        "mapping": null,
        "reasoning": "Room labels are distinct identifiers with no order."
    },
    "prioridad": {
        "subclass": "ORDINAL",
        "mapping": {"Baja": 0, "Media": 1, "Alta": 2, "Critica": 3},
        "reasoning": "Clear severity escalation from Baja to Critica."
    }
}
}
}

```

7. Ensamblado del TOON Dossier

Se combinan todos los metadatos previamente calculados (**fichero JSON de la etapa 1.6**) en un TOON. Esta es la única entrada a la etapa 2. A continuación se muestran ejemplos completos del payload de TOON que será enviados al LLM.

TOON - UN SOLO TARGET

```

# GLOBAL_METADATA
total_rows: 1000000
targets: [sala]
rare_goups_excluded: []

```

```
sampling_strategy: Stratified_SingleTarget
search_config:
  search_strategy : auto
  max_iter : 50
  max_combinations : 200
  cv_folds : 10
  timeout_minutes : 30 # CAREFUL, MAYBE HIGHER VALUES (OR DELETE)

# REPRESENTATIVE_SAMPLE_100
| id_emp | departamento | antigüedad | teletrabajo | sala |
| 1024 | Ventas | 5 | Si | Sala_A |
| 2055 | IT | 2 | No | Sala_B |
... (97 rows remaining)

# CATEGORICAL_SUBCLASS_EVALUATION
departamento:
  technical_level: 2
  is_target: False
  ig_scores: {sala: 0.38}
  ig_transform_applied': none | log1p | yeo-johnson
  null_ratio: 0.02
  unique_pool: [IT, Ventas, RRHH, Finanzas]
  user_mandatory: False
cargo_nivel:
  technical_level: 2
  is_target: False
  ig_scores: {sala: 0.12}
  ig_transform_applied': none | log1p | yeo-johnson
  null_ratio 0.00
  unique_pool: [Junior, Middle, Senior, Manager, Director]
  user_mandatory: True
  llm_instruction': 'This feature is statistically irrelevant, but the
                    user requires its inclusion for business reasons.
                    Generate code that includes it in every stage of
                    the pipeline.'
```

TARGET_SUBCLASS_EVALUATION

```
sala:
  technical_level: 2
  is_target: True
  ig_scores: {} # targets have no IG score against themselves
  ig_transform_applied: none | log1p | yeo-johnson
  unique_pool: [H, NH, O]
  user_mandatory: False
```

SEMANTIC_CLASSIFICATION_RESULTS

```
departamento:
  subclass: NOMINAL
  mapping: null
  reasoning: Corporate departments are distinct entities with no natural
hierarchy.

teletrabajo:
  subclass: ORDINAL
  mapping: {No: 0, Si: 1}
  reasoning: Boolean-like categorical with a logical progression from absence to
presence.
```

```
# Target classification
sala:
  subclass: NOMINAL
  mapping: null
  reasoning: Room labels are distinct identifiers with no natural ordering.
```

TOON - MÚLTIPLES TARGETS

```
# GLOBAL_METADATA
total_rows: 1000000
targets: [sala, prioridad]
sampling_strategy: Stratified_MultiTarget_Combined
rare_groups_excluded: [NH_C] # group too rare, sacrificed to keep
n=100

# REPRESENTATIVE_SAMPLE_100
| id_emp | departamento | cargo_nivel | turno | sala | prioridad |
| 5021   | IT           | Senior      | Noche | Sala_C | Alta      |
| 1092   | Ventas       | Junior      | Tarde | Sala_A | Baja      | ... (97
rows remaining)

# CATEGORICAL_SUBCLASS_EVALUATION
departamento:
  technical_level: 2
  ig_scores: {sala: 0.42, prioridad: 0.05}
  ig_transform_applied: none | log1p | yeo-johnson
  unique_pool: [IT, Ventas, RRHH, Finanzas]
  user_mandatory: False

cargo_nivel:
  technical_level: 2
  ig_scores: {sala: 0.15, prioridad: 0.82}
  ig_transform_applied: none | log1p | yeo-johnson
  unique_pool: [Junior, Middle, Senior, Manager, Director]
  user_mandatory: False

turno:
  technical_level: 2
  ig_scores: {sala: 0.65, prioridad: 0.10}
  ig_transform_applied: none | log1p | yeo-johnson
  unique_pool: [Mañana, Tarde, Noche]
  user_mandatory: True # Shielded

# TARGET_SUBCLASS_EVALUATION (Type 2 targets – needed by Stage 2)
sala:
  technical_level: 2
  is_target: True
  ig_scores: {} # targets have no IG score against themselves
  ig_transform_applied: none
  unique_pool: [Sala_A, Sala_B, Sala_C]
  user_mandatory: False

prioridad:
  technical_level: 2
  is_target: True
  ig_scores: {}
  ig_transform_applied: none
  unique_pool: [Baja, Media, Alta, Critica]
```

user_mandatory: False

SEMANTIC_CLASSIFICATION_RESULTS

departamento:

subclass: NOMINAL

mapping: null

reasoning: Department names have no inherent mathematical order.

cargo_nivel:

subclass: ORDINAL

mapping: {Junior: 0, Middle: 1, Senior: 2, Manager: 3, Director: 4}

reasoning: Professional hierarchy detected. High IG for 'prioridad' confirms structured influence.

turno:

subclass: ORDINAL

mapping: {Manana: 0, Tarde: 1, Noche: 2}

reasoning: Temporal sequence detected. High IG for 'sala' suggests shift-based allocation.

Target classification

sala:

subclass: NOMINAL

mapping: null

reasoning: Room labels are distinct identifiers with no order.

Stage 2 will use ClassifierChain with F1/AUC-ROC metrics.

prioridad:

subclass: ORDINAL

mapping: {Baja: 0, Media: 1, Alta: 2, Critica: 3}

reasoning: Clear severity escalation from Baja to Critica.

Stage 2 will use LGBMClassifier with linear-weighted Kappa metric.

ETAPA 2: INTELLIGENT PIPELINE ORCHESTRATION

Esta etapa utiliza el TOON Dossier del Stage 1 para construir y ejecutar un pipeline completo de entrenamiento de ML. El LLM selecciona la arquitectura y la estrategia de búsqueda de hiperparámetros, y genera el código. Si la ejecución falla, un bucle de autorreparación intenta corregir automáticamente.

1. Matriz de dependencia entre variables.

Solo se ejecuta con dos o más targets. Evalúa la dependencia entre todos los pares de targets y devuelve una matriz simétrica de *scores* con valores en [0, 1].

	Ambas categóricas	Ambas numéricas	Mezcladas
Test	χ^2	<i>Spearman</i>	<i>Kruskal-Wallis</i>
Normalización	Cramér's $V \rightarrow [0,1]$	$ \text{coef.} \rightarrow [0,1]$	$\text{Eta}^2 \rightarrow [0,1]$
Independencia	$V < 0.10$	$ r < 0.30$	$\eta^2 < 0.06$
Dependencia	$V \geq 0.10$	$ r \geq 0.3$	$\eta^2 \geq 0.06$

Nota:

Si hay más de dos targets hay que hacerlo por pares: target_1->target_2, target_1->target_3, target_2->target_3

```
def cramers_v(x, y) -> float:
    tabla = pd.crosstab(x, y)
    chi2, _, _, _ = chi2_contingency(tabla)
    n = tabla.sum().sum()
    phi2 = chi2 / n
    r, c = tabla.shape
    return np.sqrt(phi2 / min(r-1, c-1))

def eta_squared(categorical, numerical) -> float:
    groups = [numerical[categorical == g]
    for g in categorical.unique()]
    stat, _ = kruskal(*groups)
    n = len(numerical)
    return stat / (n - 1)

def dependency_matrix(df, targets, tipos) -> pd.DataFrame:
    matrix = pd.DataFrame(1.0, index=targets, columns=targets)
    for i, t1 in enumerate(targets):
        for t2 in targets[i+1:]:
            if tipos[t1] in (1,2) and tipos[t2] in (1,2):
                score = cramers_v(df[t1], df[t2])
            elif tipos[t1] in (3,4) and tipos[t2] in (3,4):
                score = abs(spearmanr(df[t1], df[t2]).statistic)
            else: # Mixed
                cat, num = (t1, t2) if tipos[t1] in (1,2) else (t2, t1)
                score = eta_squared(df[cat], df[num])
            matrix.loc[t1, t2] = matrix.loc[t2, t1] = round(score, 4)
    return matrix
```

Teniendo en cuenta el dataset hospitalario, asumiendo que los tres targets (especialidad, sala y prioridad) son categóricos (tipo 2). Se aplica χ^2 + Cramér's V:

```
# Result of dependency_matrix
              especialidad  sala  prioridad
especialidad           1.00    0.82    0.79
sala                   0.82    1.00    0.67
prioridad              0.79    0.67    1.00

# Interpretation:
# especialidad → sala: V = 0.82 (strong dependency – gating candidate)
# especialidad → prioridad: V = 0.79 (strong dependency – gating candidate)
# sala → prioridad: V = 0.67 (moderate dependency)
# All scores >> threshold (0.1) → chain strategy applies.
# Additionally: detect_structural_absences() will flag 'especialidad' #
as gating target because sala and prioridad are NaN whenever #
especialidad = 'otros' → GatedChain overrides ClassifierChain.
```

2. Definición del orden de orquestación de modelos

Se utiliza la matriz de dependencia para decidir cómo modelar múltiples *targets*. La decisión depende del valor del *threshold* y del tipo de los targets.

El parámetro *threshold* es el valor mínimo de dependencia entre cualquier par de targets por encima del cual se considera que existe una relación suficiente para usar una estrategia de cadena en lugar de MultiOutput. Si $max_dependency < threshold$, los targets se consideran independientes. Se puede configurar de dos modos:

- Valor fijo: valor conservador recomendado como punto de partida (basado en el umbral mínimo de V de Cramér-Cohen). Si la dependencia más alta supera 0.1, se activa la cadena.
- Modo automático: el umbral se calcula como la *mediana + desviación típica* de todos los scores de la matriz (excluyendo la diagonal). Esto adapta el umbral al rango real de dependencias del dataset: si todos los scores son bajos, el umbral se reduce proporcionalmente y no fuerza cadenas innecesarias.

```
def compute_threshold(matrix: pd.DataFrame, threshold_cfg) -> float:
    """Resolves threshold from config: fixed float or automatic."""
    scores= matrix.values[~np.eye(len(matrix), dtype=bool)] #exclude diag
    if threshold_cfg == 'auto':
        return float(np.median(scores) + np.std(scores))
    return float(threshold_cfg) # user-defined fixed value
```

ÁRBOL DE DECISIÓN DE ESTRATEGIAS

La prioridad de evaluación es siempre: **GatedChain** → **MultiOutput** → **tipo de targets** → **ClassifierChain/RegressorChain/HybridChain**. Dentro de **ClassifierChain**, la subclase ordinal de los targets determina el modelo concreto:

- **Ausencias Estructurales Detectadas** → GatedChain (prioridad máxima)
- **$max_dependency < threshold$** → MultiOutput (targets independientes, orden irrelevante)
- **Todos numéricos** → RegressorChain (ordenados por score de influencia descendente)
- **Tipos mixtos** → HybridChain (predicciones del primer target se añaden como features del siguiente; usar `cross_val_predict` para evitar leakage)
- **Todos categóricos** → ClassifierChain (el modelo concreto por target depende de si es Nominal u Ordinal)
 - Target Nominal → *perfil: subclass=NOMINAL, n_classes=N* → el LLM elige modelo de clasificación estándar y métrica apropiada
 - Target Ordinal → *perfil: subclass=ORDINAL, n_classes=N, mapping={...}* → el LLM elige modelo y métrica respetando la estructura ordinal

GATED CHAIN

Las estrategias anteriores asumen que todos los targets tienen valores en todas las filas. Pero existe una situación que ninguna de ellas modela correctamente: cuando la ausencia de valor en un target no es aleatoria, sino que está estructuralmente determinada por el valor de otro target.

Ejemplo: dataset con tres targets — especialidad (msk / otros), sala (H / NH) y prioridad (A / B / C). Cuando la *especialidad* = *otros*, los campos sala y prioridad siempre están vacíos, no

porque falten datos, sino porque no aplican. Este patrón se denomina Missing Not At Random (MNAR).

¿Por qué un ClassifierChain falla aquí? Porque intentaría predecir la sala y la prioridad de todos los registros, incluidos los de otros, lo que generaría predicciones sin sentido. Además, la matriz de dependencia devolvería una correlación artificialmente alta entre especialidad y los otros targets, precisamente porque los NaN se covariarían perfectamente con el valor 'otros'. La solución correcta es una arquitectura Gating + Specialist: un modelo maestro (gating model) clasifica primero el target raíz. Solo cuando el resultado activa el subconjunto relevante, se ejecutan los modelos especializados, entrenados exclusivamente sobre ese subconjunto. Los registros que no activan el gate reciben una salida nula o una etiqueta especial (N/A), nunca una predicción forzada.

Nota: Si el sistema no detecta esta estructura y aplica ClassifierChain o HybridChain, el modelo entrenará mezclando filas con datos reales y filas con ausencias estructurales, lo que le hará aprender un patrón espurio. Las métricas de validación parecerán aceptables (porque el modelo aprenderá que 'sala vacía' covaría con 'especialidad=otros'), pero el modelo será inútil en producción para los casos que realmente importan. Detectarlo antes de enviar el dossier al LLM orquestador es la única forma de que el script generado sea correcto.

```
def detect_structural_absences(df: pd.DataFrame, targets: list[str],
                              umbral: float = 0.95) -> dict | None:
    """
    Detects if the absence of one target co-varies structurally with
    the value of another (Missing Not At Random – MNAR).
    For each target t1 with missing values, checks whether other
    targets are also missing in >= umbral of those same rows.
    Returns {'gating': t1, 'dependents': [...]} or None.
    """

    for t1 in targets:
        mask = df[t1].isna() | (df[t1].astype(str).str.strip() == '')
        if mask.sum() == 0:
            continue # t1 has no missing values – cannot be gating target
        dependents = [
            t2 for t2 in targets
            if t2 != t1 and (df.loc[mask, t2].isna()).mean() >= umbral
        ]
        if dependents:
            return {'gating': t1, 'dependents': dependents}
    return None

def build_chain_strategy(df, matrix, tipos, threshold_cfg, toon_dossier) -> dict:
    """
    Determines chain strategy – GatedChain takes priority over all others.
    """

    targets = matrix.index.tolist()
    threshold = compute_threshold(matrix, threshold_cfg)

    # 1. Structural absence check – overrides all other strategies
    gated = detect_structural_absences(df, targets)
    if gated:
        subset_df = df[df[gated['gating']].notna()]
```

```

sub_matrix = dependency_matrix(subset_df, gated['dependents'], tipos)
return {
    'strategy': 'GatedChain',
    'gating': gated['gating'],
    'dependents': gated['dependents'],
    'sub_strategy': build_chain_strategy(
        subset_df, sub_matrix, tipos, threshold_cfg
    )
}

# 2. Independent targets
max_dep = matrix.where(matrix < 1.0).max().max()
if max_dep < threshold:
    return {'strategy': 'MultiOutput', 'order': targets}

influence = matrix.where(matrix < 1.0).sum(axis=1)
order = influence.sort_values(ascending=False).index.tolist()
todos_cat = all(tipos[t] in (1, 2) for t in targets)
todos_num = all(tipos[t] in (3, 4) for t in targets)

# 3. All numerical
if todos_num:
    return {'strategy': 'RegressorChain', 'order': order}
# 4. Mixed types
if todos_cat:
    return {'strategy': 'ClassifierChain', 'order': order}

# 5. All categorical – provide statistical profile per target for the LLM
# The LLM decides model and metric; code only provides the facts.
semantic = toon_dossier['semantic_classification']
target_profiles = {
    t: {
        'subclass': semantica.get(t, {}).get('subclass', 'NOMINAL'),
        'n_classes': int(df[t].nunique()),
        'mapping': semantica.get(t, {}).get('mapping', None)
    }
    for t in order
}
return { 'strategy': 'ClassifierChain',
        'order': order,
        'target_profiles': target_profiles
}

```

3. Definición del espacio de búsqueda de hiperparámetros

El sistema utiliza un diccionario de espacios predefinidos para los modelos más comunes. El LLM orquestador los recibe como base y puede ajustar rangos si el contexto estadístico lo justifica, pero siempre respetando los límites de search_config.

```

DEFAULT_SEARCH_SPACES = {
    # — CLASIFICACIÓN —
    "LGBMClassifier": {
        "n_estimators" : [100, 300, 500],
        "learning_rate" : [0.01, 0.05, 0.1],
        "max_depth" : [3, 5, 7, -1],
        "subsample" : [0.7, 0.85, 1.0],
        "num_leaves" : [31, 63, 127],
    }
}

```

```

    },
    "RandomForestClassifier": {
        "n_estimators" : [100, 300, 500],
        "max_depth" : [None, 5, 10, 20],
        "min_samples_split": [2, 5, 10],
        "max_features" : ["sqrt", "log2"],
    },
    "XGBClassifier": {
        "n_estimators" : [100, 300, 500],
        "learning_rate" : [0.01, 0.05, 0.1],
        "max_depth" : [3, 5, 7],
        "subsample" : [0.7, 0.9, 1.0],
        "colsample_bytree": [0.7, 0.9, 1.0],
    },
    # — REGRESIÓN —
    "LGBMRegressor": {
        "n_estimators" : [100, 300, 500],
        "learning_rate" : [0.01, 0.05, 0.1],
        "max_depth" : [3, 5, 7, -1],
        "subsample" : [0.7, 0.85, 1.0],
        "num_leaves" : [31, 63, 127],
    },
    "RandomForestRegressor": {
        "n_estimators" : [100, 300, 500],
        "max_depth" : [None, 5, 10, 20],
        "min_samples_split": [2, 5, 10],
        "max_features" : ["sqrt", "log2", None], # None = all feat
    },
    "XGBRegressor": {
        "n_estimators" : [100, 300, 500],
        "learning_rate" : [0.01, 0.05, 0.1],
        "max_depth" : [3, 5, 7],
        "subsample" : [0.7, 0.9, 1.0],
        "colsample_bytree": [0.7, 0.9, 1.0],
    },
}

```

Cuando `search_strategy = "auto"`, el LLM recibe el contexto estadístico del TOON (`n_features`, `n_rows`, `n_targets`, `chain_strategy`) y aplica las siguientes heurísticas:

Condición	Método	Razonamiento
<code>n_features < 20 AND n_rows < 50k</code>	grid	<i>Espacio pequeño — exploración exhaustiva factible</i>
<code>n_features ≥ 20 OR n_rows ≥ 50k</code>	random	<i>Espacio grande — muestreo aleatorio más eficiente</i>
<code>n_targets > 1 OR GatedChain / HybridChain</code>	optuna	<i>Pipeline complejo — optimización bayesiana con pruning</i>

4. Generación del script de entrenamiento

Creación del `system_prompt` y `user_prompt`:

```

orchestrator_system_prompt = """
You are the Master Pipeline Orchestrator, a Senior Lead Data Scientist.

```

Your goal is to generate a professional, high-performance, and IMMEDIATELY EXECUTABLE Python script for model training.

DIVISION OF RESPONSIBILITIES:

The dossier already contains 'chain_strategy' with the structural pipeline decision (ClassifierChain, RegressorChain, HybridChain, GatedChain, MultiOutput). YOUR responsibility is to decide the MODELS and METRICS for each target.

RESPONSIBILITIES:

1. Read 'chain_strategy.target_profiles' for each target:

- subclass: NOMINAL or ORDINAL
- n_classes: number of unique classes
- mapping: ordinal encoding if applicable

Select the most appropriate model and evaluation metric based on these facts.

2. Implement the chain strategy exactly as specified in 'chain_strategy.strategy'.

Do NOT change the strategy or execution order.

3. HybridChain / GatedChain: ALWAYS use cross_val_predict() –

NEVER call predict() or predict_proba() on the full training set.

4. Use the search_type specified in the dossier for hyperparameter tuning.

5. Save each model as model_{target}.joblib using joblib.

6. Print final summary with best params and validation metrics.

HYPERPARAMETER SEARCH SPACE RULES:

1. If the model EXISTS in DEFAULT_SEARCH_SPACES, use it as base. You may adjust ranges based on TOON context, but always respect:

- grid search : total combinations <= search_budget.max_combinations
- random search: use scipy.stats distributions, not discrete lists
- optuna : define suggest_int / suggest_float / suggest_categorical

2. If the model does NOT exist in DEFAULT_SEARCH_SPACES:

- a. Include ONLY the 3-5 most impactful hyperparameters for that model.

Prioritize: complexity control, regularization, learning rate.

- b. Use no more than 3 values per hyperparameter.

- c. Verify parameter names against the sklearn-compatible API.

- d. Add a comment next to each parameter explaining why it was included.

3. If search_strategy is "auto", select method using:

- n_features < 20 AND n_rows < 50k → grid
- n_features >= 20 OR n_rows >= 50k → random
- n_targets > 1 OR GatedChain → optuna

4. Always respect search_config, timeout_minutes and cv_folds.

5. Print param_space as a dict BEFORE the search starts.

This makes the search space auditable in the execution log.

Example: print("param_space:", param_space)

OUTPUT RULES:

- Provide a brief "Orchestration Reasoning" section.
- Provide the Python Code block.
- Do not include any conversational text after the code.
- Ensure all necessary imports (pandas, sklearn, joblib, etc.) are at the top of the generated script.

"""

```
user_prompt = f>Please orchestrate the training pipeline for the following
dossier:\n\n{toon_dossier}"
```

```

payload = {
    "model": MODEL_NAME,
    "messages": [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ],
    'options': {'temperature': 0.2, 'num_ctx': 4096} #Same as previous
}

```

5. Ejecución en Sandbox: Subproceso vs Docker

El sistema detecta automáticamente si Docker está disponible y usa contenedor por defecto. Si no, hace la llamada usando un subproceso. El usuario puede forzar cualquiera de los dos modos en el fichero manifest (execution_mode).

	Subproceso	Docker
Aislamiento	Bajo - comparte Python y FS del host	<i>Alto - entorno completamente aislado</i>
Seguridad	El script puede leer/escribir paths del host	<i>Solo ve dataset (read-only) y carpeta de salida</i>
Reproducibilidad	Depende del entorno local	<i>100% reproducible en cualquier máquina</i>
Dependencias	Falla si falta algo en el host	<i>Se instalan dentro del contenedor</i>
Overhead	~0 ms	<i>2-5 s de arranque (imagen cacheada)</i>
Debugging	logs directos en consola	<i>docker logs / volumen montado</i>
Requisito	Ninguno	<i>Docker instalado y en ejecución</i>

```

def _docker_available() -> bool:
    try:
        r = subprocess.run(['docker', 'info'],
                           capture_output=True, timeout=5)
        return r.returncode == 0
    except (FileNotFoundError, subprocess.TimeoutExpired):
        return False

def _run_in_subprocess(code: str) -> subprocess.CompletedProcess:
    with open('/tmp/script_automl.py', 'w') as f:
        f.write(code)
    return subprocess.run(
        ['python3', '/tmp/script_automl.py'],
        capture_output=True, text=True, timeout=900
    )

def _run_in_docker(code: str,
                  dataset_path: str,
                  output_path: str) -> subprocess.CompletedProcess:
    with open('/tmp/script_automl.py', 'w') as f:
        f.write(code)
    cmd = [
        'docker', 'run', '--rm',
        '-v', f'{dataset_path}:/data:ro',          # dataset read-only
        '-v', f'{output_path}:/output',           # models output
        '-v', '/tmp/script_automl.py:/app/run.py', # script
        'python:3.11-slim',
        'bash', '-c',
        'pip install -q scikit-learn lightgbm xgboost joblib optuna '
        '&& python /app/run.py'
    ]
    return subprocess.run(cmd, capture_output=True, text=True, timeout=960)

def run_generated_code(code: str,
                      dataset_path: str,
                      output_path: str,
                      mode: str = manifest['execution_mode']) ->
subprocess.CompletedProcess:
    """
    mode: 'auto'      → Docker si disponible, subprocess si no
         'docker'    → fuerza Docker (error si no disponible)
         'subprocess' → fuerza subprocess
    """
    use_docker = _docker_available() if mode == 'auto' else mode == 'docker'
    if use_docker:
        return _run_in_docker(code, dataset_path, output_path)
    return _run_in_subprocess(code)

```

El script generado se ejecuta en un entorno aislado. Si falla, el bucle de auto-healing clasifica el error y solicita una reparación dirigida al LLM de código. Los errores se clasifican antes de enviarlos al LLM de reparación. Errores de sintaxis/import (intento 1) reciben la instrucción de cambio mínimo. Errores estructurales (intento 2+) permiten reescribir la arquitectura. Errores irrecuperables (OOM, disco lleno) abortan sin reintentar.

SELF-HEALING SYSTEM PROMPT

```

self_healing_system_prompt = self_healing_system_prompt = """
You are the Master Pipeline Orchestrator in DEBUG MODE.
Your previous code failed, and you must now act as a Senior Debugger.

CONTEXT:
The original script was generated by an orchestrator that already
decided:
  - The chain strategy (ClassifierChain, GatedChain, etc.)
  - The model and metric for each target based on its subclass and
    n_classes
  - The hyperparameter search space and CV strategy
These decisions belong to the orchestrator, not to you.
Do NOT revisit or replace them unless the error log explicitly points to
them.

REPAIR RULES:
1. Fix ONLY what the error log points to.
   syntax / import errors → minimal one-line fix, no restructuring.
   shape / mismatch errors → fix the transformation causing the
                           mismatch.
   structural errors → may rewrite the affected block, but preserve
                       model choices, metric choices, and chain order.
2. Preserve model and metric choices: if the original script uses
   LGBMClassifier with kappa_linear for an ordinal target, keep it.
   Only replace a model if the error is caused by that specific model.
3. NEVER replace cross_val_predict() with predict() or predict_proba()
   on the full training set – this would introduce target leakage.
4. NEVER remove joblib persistence blocks.
5. On attempt 2+, you may rewrite larger blocks if the error is
   structural, but the chain strategy and execution order must remain
   identical.

UNRECOVERABLE ERRORS:
If the error is MemoryError, CUDA out of memory, SystemExit, or
OSError Errno 28 (disk full): return the token UNRECOVERABLE_ERROR
on the first line, followed by a one-line explanation.

OUTPUT:
Return ONLY the corrected Python code block.
No apologies, no explanations unless the fix is a critical
architecture change.
"""

```

```

import subprocess, re

UNRECOVERABLE = ['MemoryError', 'CUDA out of memory',
                  'SystemExit', 'OSError: [Errno 28]']
MAX_ATTEMPTS = 3

def classify_error(stderr: str) -> str:
    if 'SyntaxError' in stderr or 'IndentationError' in stderr: return 'syntax'
    if 'ModuleNotFoundError' in stderr or 'ImportError' in stderr: return 'import'

```

```

    if any(k in stderr for k in ['shape', 'dimension', 'mismatch']): return
    'shape'
    return 'structural'

def run_in_sandbox(code: str) -> subprocess.CompletedProcess:
    with open('/tmp/script_automl.py', 'w') as f:
        f.write(code)
    return subprocess.run(['python3', '/tmp/script_automl.py'],
                          capture_output=True, text=True, timeout=900)

def build_repair_prompt(dossier, broken_code, stderr, attempt, error_type) ->
str:
    return f'''
    ATTEMPT {attempt} of {MAX_ATTEMPTS}. Error type: {error_type}.
    ERROR LOG: {stderr} ORIGINAL CODE: {broken_code}
    ORIGINAL DOSSIER: {dossier}
    Fix ONLY what the error log points to.
    Preserve chain strategy, model choices, and metric choices.
    Return ONLY the corrected Python code block. '''

def call_coding_llm(system_prompt, user_prompt,
                    ollama_url, coding_model) -> str:
    payload = {
        'model': coding_model,
        'messages': [
            {'role': 'system', 'content': system_prompt},
            {'role': 'user', 'content': user_prompt}
        ],
        'options': {'temperature': 0.1, 'num_ctx': 4096}
    }
    # temperature 0.1: deterministic repair – no creativity needed
    # num_ctx: should use get_safe_num_ctx() as in Step 1.4
    resp = requests.post(ollama_url, json=payload)
    resp.raise_for_status()
    return resp.json()['message']['content']

def self_healing_loop(dossier: str, initial_code: str,
                     ollama_url: str, coding_model: str) -> dict:
    current_code = initial_code
    for attempt in range(1, MAX_ATTEMPTS + 1):
        result = run_in_sandbox(current_code)
        if result.returncode == 0:
            return {'status': 'success',
                    'attempts': attempt,
                    'output': result.stdout}
        if any(e in result.stderr for e in UNRECOVERABLE):
            return {'status': 'unrecoverable',
                    'attempts': attempt, 'error': result.stderr}
        if attempt == MAX_ATTEMPTS:
            break
        error_type = classify_error(result.stderr)
        repair_prompt = build_repair_prompt(
            dossier, current_code, result.stderr, attempt, error_type)
        current_code = call_coding_llm(self_healing_system_prompt,
                                       repair_prompt, ollama_url, coding_model)
    return {'status': 'failed',
            'attempts': MAX_ATTEMPTS,
            'error': result.stderr}

```

Si se ejecuta de forma correcta, se guardarían los siguientes ficheros en disco:

- `model_{nombre_target}.joblib` — modelo entrenado para cada target
- `preprocessor_stage{N}.joblib` — preprocesador ajustado para cada etapa
- Resumen en consola: mejores hiperparámetros + métricas de validación (F1, Kappa, MAE, AUC-ROC)